KFORCE PYTHON DEVELOPER

Interview Preparation Guide

Python • SQL • REST/SOAP • Enterprise Integration • CI/CD • GenAI

Goal: present strong Python/backend/SQL/REST experience clearly, while handling Django, SOAP/WSDL, Angular, Azure DevOps, embeddings and agent-framework gaps truthfully.

Anonymous role-centric preparation • 2026-09-18

1. Target Role & Match Overview

Role: Python Developer, Kalyani Nagar, Pune. Requested experience: 4-7 years. The strongest verified overlap is Python, SQL, REST APIs, database optimization, Git, CI/CD concepts, enterprise integrations, backend application development and applied LLM/prompt workflows.

- Strong: Python, Flask/FastAPI, SQL/MySQL, REST APIs, Git, backend debugging, database/query optimization, enterprise API integrations, AWS-based DevOps workflows.
- Partial: CI/CD pipelines, application architecture, GenAI/GPT APIs, prompt engineering, RAG concepts, scalable cloud workflows.
- Gap: professional Django, SOAP/WSDL, Angular, Azure DevOps, dedicated embeddings/agent-framework production work, and the requested 4-7 year tenure.

Interview positioning: “I match the Python, SQL, REST, integration and backend problem-solving core. My direct framework/cloud experience is Flask/FastAPI and AWS; I can transfer those patterns to Django and Azure DevOps without claiming experience I have not used professionally.”

2. High-Priority Keyword Map

Priority 1 - must be fluent

- Python: OOP, exceptions, context managers, decorators, generators, typing, virtual environments, packaging, debugging.
- SQL: joins, indexes, execution plans, normalization, transactions, stored procedures, query tuning.
- REST: HTTP methods/status codes, validation, auth, pagination, idempotency, versioning, error contracts.

Priority 2 - JD-specific concepts

- Django architecture: project/app structure, URL routing, views, serializers, ORM, middleware, migrations, authentication.
- SOAP/WSDL and SOA: contract-first services, XML envelopes, operations, schemas, service boundaries.
- Azure DevOps: Repos, Pipelines, Boards, artifacts; explain by mapping from Git + CI/CD experience.

Priority 3 - GenAI

- GPT/LLM API integration, embeddings concept, vector retrieval concept, prompt design, evaluation, guardrails, tool/agent loops.

3. Gap Handling - Exact Interview Language

Django

“My professional Python web work has mainly been Flask/FastAPI rather than Django. I already work with routing, validation, authentication, ORM/database patterns, REST services and testing. I understand Django’s MVT structure, ORM, migrations and middleware conceptually, and I can ramp up quickly without claiming production Django experience.”

SOAP/WSDL

“My integrations have mainly been REST/Oracle ORDS. I understand SOAP as XML-based contract-driven messaging and WSDL as the formal service description. I would use the WSDL/schema to generate or validate client bindings, then handle auth, timeouts, retries and fault responses.”

Angular / Azure DevOps

“My frontend work is React, not Angular, and my DevOps experience is Git/CI-CD with AWS rather than Azure DevOps. The transferable concepts are component-based UI integration, source control, build/test pipelines, deployment gates and environment configuration.”

4. 60-Second Introduction

“I am a Python developer and data engineer with 3+ years of experience building backend APIs, SQL-driven business applications, cloud workflows and enterprise integrations. In my earlier software role I built Flask-based REST APIs, authentication and workflow modules with MySQL and React, and I worked on query optimization using indexes, stored procedures and views. More recently I have built Python/AWS automation and data workflows integrating Oracle REST APIs, Lambda, Step Functions and monitoring. I also have practical exposure to LLM APIs, prompt engineering and RAG concepts. For this role, my strongest match is Python, SQL, REST, Git, integration and backend troubleshooting, while Django, SOAP/WSDL, Angular and Azure DevOps are areas I am prepared to ramp up on.”

5. 2-Minute Introduction

Use a four-part structure: **experience** → **backend proof** → **cloud/integration proof** → **JD fit/gaps**.

- Experience: 3+ years across Python software development and data/backend engineering.
- Backend proof: Flask/FastAPI, REST APIs, MySQL, authentication, business workflow modules, React integration, testing and production issue resolution.
- Cloud/integration proof: Python AWS workflows, Oracle ORDS/REST APIs, Lambda, S3, SQS, Step Functions, Glue, CloudWatch, validation and failure handling.
- GenAI proof: practical LLM API/prompt automation and RAG concepts - describe exactly what you built or tested, not unsupported model-training claims.

Close with: “The role is a strong match on Python/SQL/API engineering. I am transparent that my direct framework stack is Flask/FastAPI and AWS, so I would treat Django and Azure DevOps as transferable-skill ramp-up areas.”

6. Python Core - Questions & Answers

List vs tuple vs set vs dict?

List: ordered/mutable sequence. Tuple: ordered/immutable record-like sequence. Set: unique unordered membership collection. Dict: key-value mapping with fast average lookup.

Mutable default argument problem?

Default values are created once at function definition. Use **None** and initialize inside the function when you need a fresh list/dict.

Generator vs list?

A generator yields values lazily and is memory-efficient for large streams; a list materializes all values immediately.

Decorator?

A function that wraps another callable to add behavior such as logging, authorization, retries or timing without changing the target function’s core code.

Context manager?

Controls setup/cleanup around a block using **with**; common for files, database connections and locks.

7. OOP & Application Architecture

Explain OOP in a backend service.

Encapsulation keeps state/behavior together; abstraction exposes stable interfaces; inheritance can reuse behavior but composition is often safer; polymorphism lets interchangeable implementations satisfy the same interface.

How would you structure a maintainable Python service?

- API/controller layer for HTTP concerns.
- Service layer for business logic.
- Repository/data-access layer for persistence.
- Schemas/models for validated contracts.
- Cross-cutting modules for configuration, logging, auth, retries and observability.

What makes an application scalable?

Stateless services where practical, horizontal scaling, efficient database access, caching, asynchronous work for slow tasks, timeouts/retries, idempotency, observability and controlled dependency usage.

8. Django - What to Know Before the Interview

Core architecture

Django follows Model-View-Template. For APIs, Django REST Framework commonly adds serializers, viewsets, routers, authentication and permissions.

- Models/ORM: model classes map to database tables; QuerySets build database queries.
- Migrations: version-controlled schema changes.
- Middleware: request/response hooks for cross-cutting behavior.
- Settings: configuration split by environment; secrets should come from secure environment/config stores.

How to answer framework comparison?

FastAPI is lightweight and type-driven with automatic OpenAPI; Django is batteries-included with ORM/admin/auth; Flask is minimal and flexible. Choose based on application needs, team conventions and ecosystem.

9. Flask/FastAPI Transfer to Django

Map what you already know instead of pretending Django tenure:

- Flask/FastAPI routes → Django URL routes/views or DRF viewsets.
- Pydantic schemas → DRF serializers/forms for validation/serialization.
- SQLAlchemy/MySQL access → Django ORM QuerySets/models.
- Auth dependencies/middleware → Django/DRF authentication, permissions and middleware.
- Pytest/API tests → Django TestCase/pytest-django/APIClient patterns.

Strong response: “Framework syntax changes, but the backend engineering concerns remain the same: contracts, validation, transactions, auth, error handling, testing, observability and database efficiency.”

10. SQL Development & Query Optimization

How do you optimize a slow SQL query?

- Inspect the execution plan and identify full scans, expensive joins, sorts or repeated subqueries.
- Add/adjust indexes based on filter/join/order patterns, not blindly.
- Select only needed columns and reduce rows early.
- Check data types, cardinality, statistics and join conditions.
- Re-test with realistic data and confirm application-level latency.

INNER vs LEFT JOIN?

INNER returns matched rows only. LEFT returns every left-side row plus matches, using NULLs where no right-side row exists.

Transaction isolation?

Isolation controls visibility of concurrent changes. Stronger isolation reduces anomalies but can increase locking/contention.

11. REST, SOAP & WSDL

REST

Resource-oriented HTTP APIs typically use JSON, standard methods/status codes, stateless requests and explicit contracts via OpenAPI or documentation.

SOAP

Protocol using XML envelopes with defined headers/body and standardized faults; commonly used in enterprise integrations requiring formal contracts and WS-* standards.

WSDL

XML document describing SOAP service operations, endpoints, messages and types. It can be used to generate strongly typed clients.

What would you focus on in either style?

- Timeouts, retries with backoff, idempotency, authentication, schema validation, correlation IDs, logging, error mapping and safe handling of sensitive payloads.

12. Enterprise Integration & SOA

What is SOA?

Service-Oriented Architecture organizes business capabilities as reusable services with explicit interfaces/contracts. Services should have clear ownership, boundaries and interoperability.

SOA vs microservices?

Both use services, but SOA often emphasizes enterprise-wide shared services/integration and may use centralized middleware; microservices usually favor smaller independently deployable services and decentralized ownership.

Integration design checklist

- Define source/target contracts and schema mapping.
- Choose sync vs async communication based on latency and resilience needs.
- Make retries safe with idempotency/deduplication.
- Add traceability: request IDs, logs, metrics, alerts and dead-letter handling.

13. Git, Azure DevOps & CI/CD

Git workflow

Use short-lived branches, small reviewed changes, meaningful commits, pull requests, automated tests and protected main/release branches.

Azure DevOps basics to know

- Repos: Git hosting and pull requests.
- Pipelines: YAML/classic build and deployment automation.
- Boards: work items/sprints.
- Artifacts: package feeds.

CI/CD pipeline stages

Checkout → dependency install → lint/static checks → unit/integration tests → package/container build → security scan → artifact publish → deploy to environment → smoke/health checks → controlled promotion/rollback.

State direct experience accurately: Git and CI/CD concepts are hands-on; Azure DevOps itself is a learning/transfer area if not used professionally.

14. GenAI - GPT, Embeddings & RAG

GPT/LLM API integration

Send structured prompts/messages, configure model parameters, validate outputs, handle rate limits/timeouts, log safely, and design fallbacks for unreliable responses.

What is an embedding?

A dense numerical vector representing semantic meaning. Similar content tends to have closer vectors under a chosen similarity metric.

RAG flow

Ingest documents → chunk → create embeddings → store/index → embed user query → retrieve relevant chunks → optionally rerank → build grounded prompt → call LLM → validate/cite output.

Important caveat

Knowing the concept is different from having production embeddings experience. Describe the exact level you have implemented or prototyped.

15. LLM Prompt Engineering

Good prompt structure

- Role/context: what the model is doing and relevant background.
- Task: explicit instruction and success criteria.
- Constraints: format, tone, exclusions, safety rules.
- Examples: few-shot examples when patterns are difficult.
- Output schema: JSON or another machine-validated structure when consumed by code.

How do you improve a weak prompt?

Create representative test cases, identify failure categories, tighten instructions/examples, constrain outputs, add retrieval/tools when needed, then compare quality systematically rather than tuning on one example.

16. Agent-Based Solutions

What is an AI agent?

A system where an LLM chooses or sequences actions/tools based on state and a goal, rather than producing only a single text completion.

Typical loop

Receive goal → inspect context/state → select tool/action → execute → observe result → update state → continue or stop.

Production safeguards

- Limit tool permissions and scope.
- Validate tool arguments and outputs.
- Set step/time/token budgets.
- Require human approval for destructive/high-impact actions.
- Persist trace logs for debugging and evaluation.

If asked about agent framework experience, separate conceptual understanding from any frameworks you have actually used.

17. Angular Gap vs React Experience

How to position the gap

“My frontend experience is React rather than Angular. I already understand component-based UI design, state/data flow, REST integration, forms, authentication flows and JavaScript/TypeScript-style application concerns. I would need to learn Angular-specific modules/components, dependency injection, RxJS and routing conventions.”

Key Angular terms to review

- Components/templates and data binding.
- Services and dependency injection.
- Routing and route guards.
- Reactive forms.
- RxJS observables and HTTP client.

18. Project Deep Dive - Business Application Platform

60-second project story

“I worked on a Python/MySQL business application with REST APIs and a React frontend. I built backend endpoints, authentication flows such as OTP/password reset, workflow modules, and integrations with internal CRM-style systems. I also optimized MySQL using stored procedures, views, functions and indexes, and supported testing and production issues.”

Likely follow-ups

- How did you design API validation and errors?
- How did you secure authentication/OTP flows?
- Give an example of a SQL query/index improvement.
- How did frontend and backend contracts stay synchronized?
- Describe one production issue and your debugging sequence - use only a real example.

19. Project Deep Dive - Serverless Enterprise Workflow

60-second project story

“I migrated a legacy finance-document workflow to Python and AWS serverless components. The solution integrated Oracle REST APIs, enriched data, generated conditional PDF/CSV outputs and stored results in S3. Lambda, SQS and other AWS services handled workflow steps, while monitoring/error handling supported operations.”

Connect to Kforce JD

- Enterprise integration: external system contract, mapping and validation.
- Application architecture: modular processing stages and clear responsibilities.
- Reliability: queues, retries, monitoring, failure handling and audit-friendly storage.
- Transferable to SOA: explicit service boundaries and API-driven integration, without claiming formal SOAP implementation.

20. Production Troubleshooting Scenarios

API suddenly returns 500

Check logs/correlation ID → reproduce → inspect recent deploy/config → validate dependency/database state → isolate failing layer → mitigate/rollback → fix → add regression test/monitor.

API latency increased

Measure endpoint timing → split application vs DB vs external dependency → inspect query plans/connection pools → check timeouts/resource saturation → optimize/cache/async as appropriate → load test.

Duplicate processing after retry

Use idempotency keys/business keys, unique constraints or dedup tables; make side effects retry-safe and record processing state.

LLM response breaks downstream parser

Request a strict schema, validate output, retry only safe failures, reject malformed responses, capture test cases, and avoid trusting free-form model text as structured data.

21. System Design - Enterprise Python Service

Prompt: design a scalable integration API

- Client → API gateway/load balancer → stateless Python service → service/business layer → database and external enterprise services.
- Use queue/background workers for long-running or failure-prone integrations.
- Store configuration/secrets securely; separate environments.
- Add cache only for suitable read-heavy/expensive lookups.
- Observability: structured logs, metrics, traces, dashboards, alerts and correlation IDs.
- Reliability: timeouts, bounded retries/backoff, circuit breaker where useful, idempotency, dead-letter queue.
- Security: authN/authZ, least privilege, input validation, TLS, secret rotation and audit logs.

22. Coding Frameworks - Python & SQL

Python problem-solving sequence

Clarify input/output → identify edge cases → choose data structure → write simple correct solution → analyze time/space complexity → test normal/empty/duplicate/error cases.

Example: frequency count

```
from collections import Counter
counts = Counter(values)
most_common = counts.most_common(1)
```

SQL framework

Define grain → identify tables/keys → filter dates/status → join deliberately → aggregate/window → handle NULL/duplicates → validate totals.

```
SELECT customer_id, SUM(amount) AS total
FROM orders
WHERE order_date >= :start_date AND order_date < :end_date
GROUP BY customer_id;
```

23. Behavioral STAR + Final Revision

Prepare 5 STAR stories

- A production issue you diagnosed and fixed.
- A SQL/API performance improvement.
- A requirement change or ambiguous integration.
- A failure/mistake and what you changed afterward.
- A case where you learned a new technology quickly.

Night-before checklist

- Practice both introductions aloud.
- Review Django request flow, ORM/migrations/middleware and DRF vocabulary.
- Review SOAP envelope/WSDL/fault concepts and REST comparison.
- Review SQL indexing/execution plans and one real optimization example.
- Review Git/CI-CD and Azure DevOps terminology.
- Review GPT, embeddings, RAG, agents and prompt evaluation at concept level.

Final rule: be specific about what you built, clear about what you only understand conceptually, and connect transferable engineering patterns to the missing stack.